

1. State whether each of the following statements are true or false. Justify your answers.

(a) When a new process is admitted in the system, the short-term scheduler must execute in order to keep the CPU busy.

(b) Partitioned multi-processor scheduling suffers from migration overheads due to data in private core-specific caches.

1a) When a new process is admitted in the system, the short-term scheduler must execute in order to keep the CPU busy.

→ False.

Justification: To keep the CPU busy, the short-term scheduler **must** execute only when a running process terminates or blocks. When a new process is admitted, it **may** be required to execute to keep the CPU busy only if there is no other running process in the system.

1b) Partitioned multi-processor scheduling suffers from migration overheads due to data in private core-specific caches.

→ False.

Justification: Migration overheads occur in global scheduling when a process partially executes on one core and then migrates to another. In partitioned scheduling processes don't migrate between cores. However, partitioned scheduling has the problem of unbalanced loading of the cores depending on the process-core mapping.

2. Consider the following set of processes that are using the same I/O device.

Process	CPU Burst Time	I/O Burst Time	Arrival Time
P1	2,8	2 (after first CPU burst)	0 (order: 1)
P2	1	0	0 (order: 2)
P3	1,1	1 (after first CPU burst)	2 (order: 1)
P4	1	0	2 (order: 2)
P5	2,3	2 (after first CPU burst)	4

(a) Draw two Gantt charts illustrating the execution of these processes using:

i. Round-Robin (quantum=2)
uniprocessor scheduling.

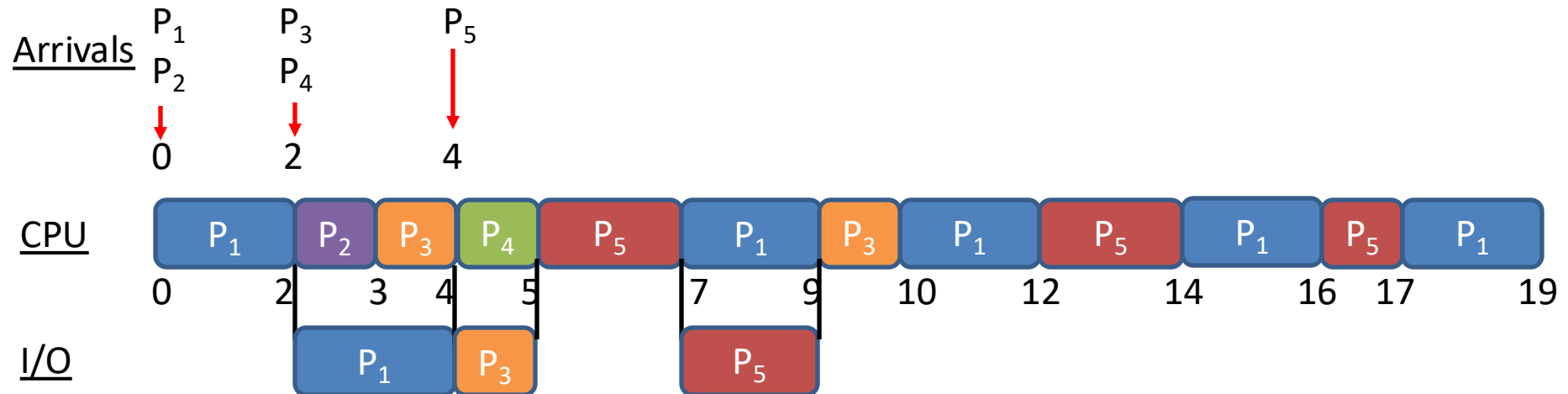
ii. Shortest Remaining Time First (SRTF)
global multi-processor scheduling with 2 cores.

Any volunteer?

Gantt Charts (uniprocessor)

Round Robin (quantum=2)

Process	CPU Burst Time	I/O Burst Time	Arrival Time
P1	2,8	2 (after first CPU burst)	0 (order: 1)
P2	1	0	0 (order: 2)
P3	1,1	1 (after first CPU burst)	2 (order: 1)
P4	1	0	2 (order: 2)
P5	2,3	2 (after first CPU burst)	4

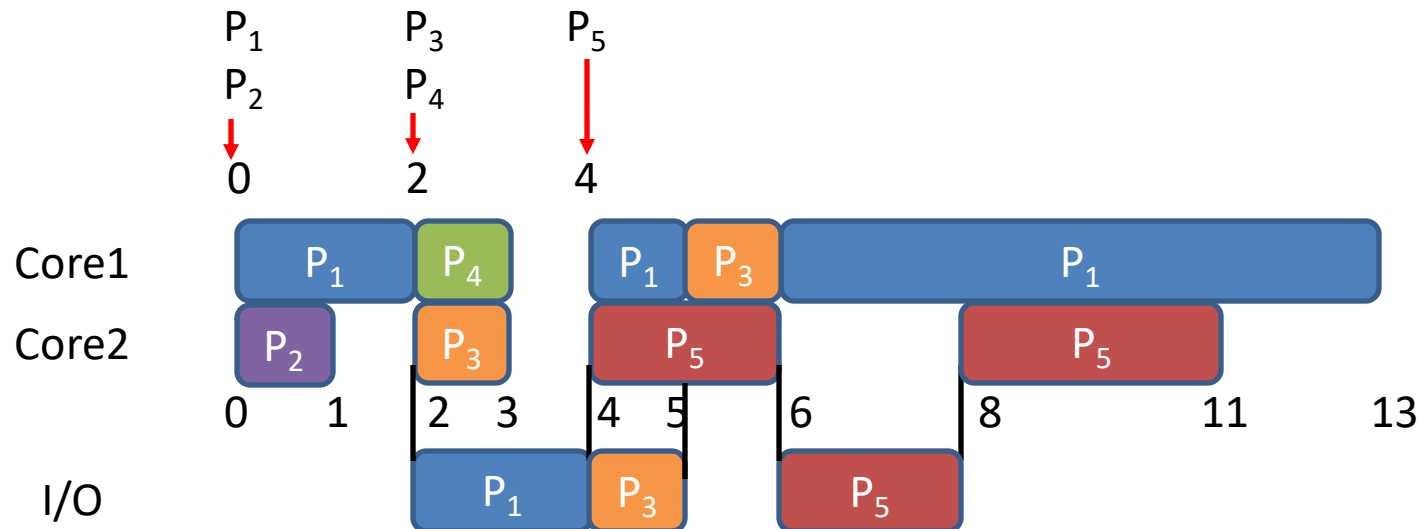


Gantt Charts (multi-processor)

Shortest Remaining Time First - SRTF
(global)

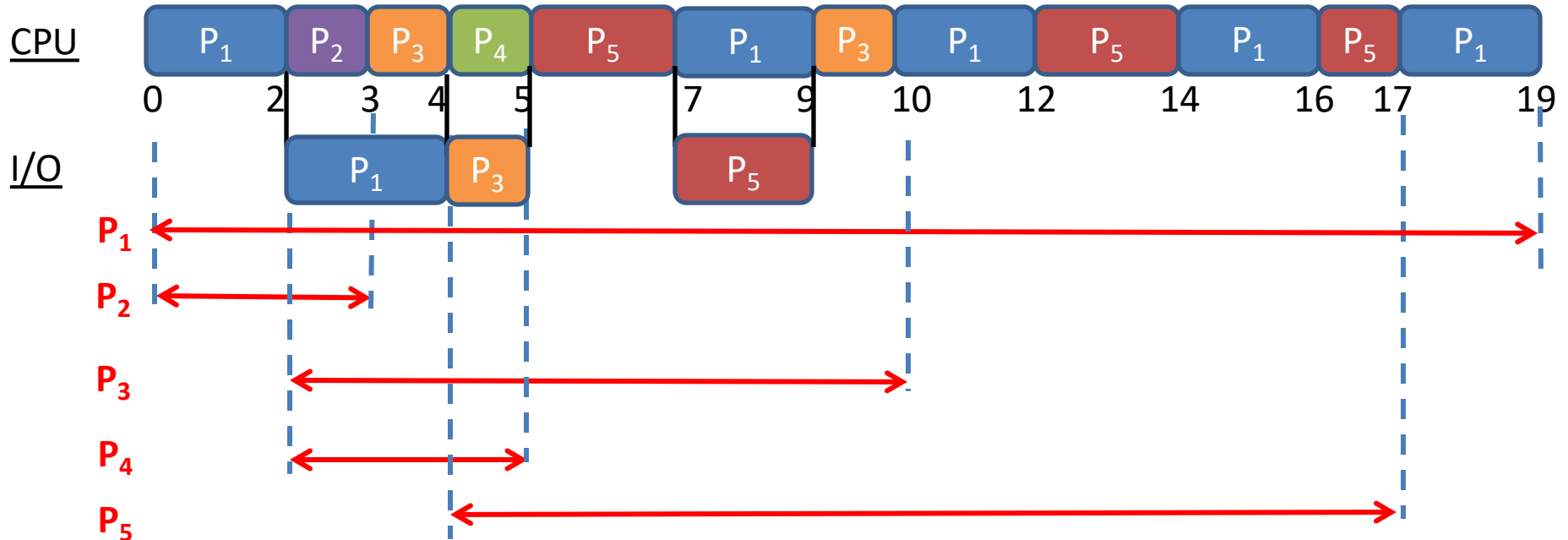
Process	CPU Burst Time	I/O Burst Time	Arrival Time
P1	2,8	2 (after first CPU burst)	0 (order: 1)
P2	1	0	0 (order: 2)
P3	1,1	1 (after first CPU burst)	2 (order: 1)
P4	1	0	2 (order: 2)
P5	2,3	2 (after first CPU burst)	4

Arrivals



Turnaround Time

Round
Robin
quantum=2



Turnaround & Waiting Times

(b) What are the turnaround and waiting times of processes for each algorithm?

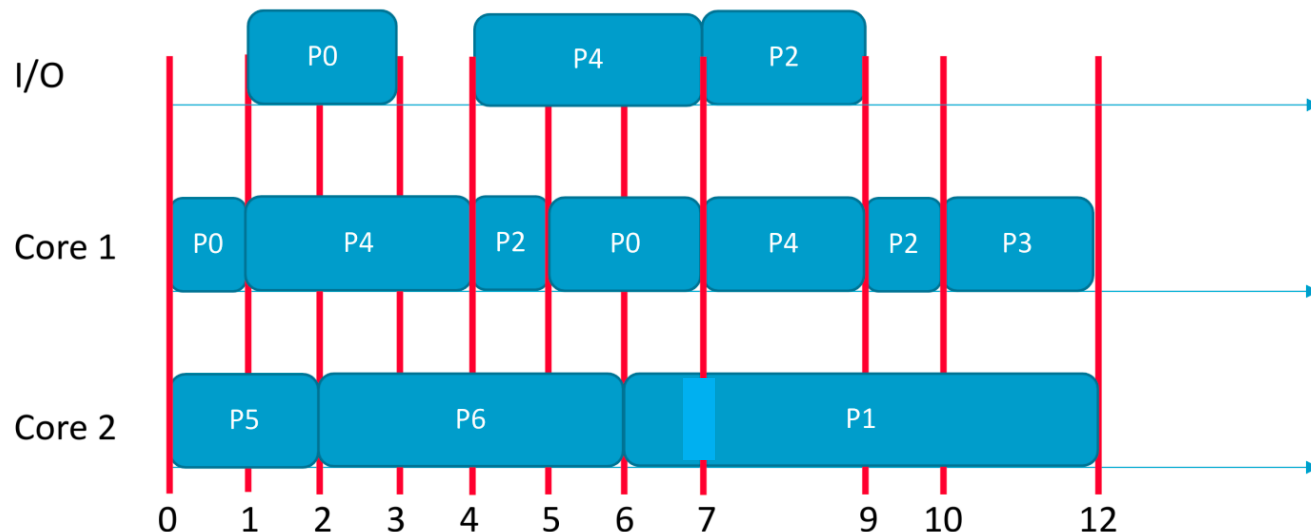
- **Turnaround Times (completion time – arrival time)**
 - Round-Robin (quantum 2): P1=19, P2=3, P3=8, P4=3, P5=13
 - SRTF(global): P1=13, P2=1, P3=4, P4=1, P5=7
- **Waiting Times (Turnaround – All bursts – time waiting for I/O)**
 - Round-Robin (quantum 2): P1=7, P2=2, P3=5, P4=2, P5=6
 - SRTF(global): P1=1, P2=0, P3=0, P4=0, P5=0

3) Consider a set of processes P0-P6 with the following arrival times.

Assume processes are added to ready queue in the sequence according to their PIDs

Process	Arrival Time Instant
P0, P1, P4, P5, P6	0 (order: P0, P1, P4, P5, P6)
P2	3
P3	9

A dual-core schedule of these processes is shown in the figure below.



3a) What are the CPU and I/O burst lengths for each process (if there are multiple CPU or I/O bursts for a process, then identify all of them).

- P0 (1, 2, 2) (CPU, I/O, CPU)
- P1 (6) (CPU)
- P2 (1, 2, 1) (CPU, I/O, CPU)
- P3 (2) (CPU)
- P4 (3, 3, 2) (CPU, I/O, CPU)
- P5 (2) (CPU)
- P6 (4) (CPU)

3b) Does the schedule represent partitioned or global multiprocessor scheduling? Identify the scheduling algorithm(s) used in the schedule (select from those covered in the lectures). Justify your answer.

There are several possible answers to this question.

- **Partitioned Scheduling (P1, P5, P6 on Core 2)**

- Core 2: Cannot be First-Come-First-Serve (FCFS) or Round Robin (RR) - P1 should have run first in these cases. The schedule can be Shortest Job First (SJF) or Shortest Remaining Time First (SRTF)
- Core 1:
 - FCFS: Not possible because P0 should have run at 4
 - If RR, quantum must be ≥ 3 , but it's not possible for the same reason above
 - SRTF and SJF are valid schedulers on Core 1 (they will all generate the schedule shown)

Round-robin also
requires P1 to be
executed first

3b)

- Global Scheduling

- FCFS or RR: Not possible because P1 should have run first in parallel with P0
- SRTF: Not possible because P6 (on core 2) should be pre-empted by P2 at time 3 if SRTF is used
- SJF: This is possible assuming SJF prioritizes based on burst lengths. The first burst length of P4 is 3 which is shorter than the burst length of P6 which is 4. Therefore at time 1, P4 is prioritized under SJF over P6.

4) Measurements of a certain system have shown that the average process runs for time T before blocking on I/O. A process switch requires time S , which is effectively wasted (overhead). Define what is meant by CPU efficiency. For round robin scheduling with quantum Q , give a formula for the CPU efficiency for each of the following cases:

(a) $Q = \infty$

(b) $Q > T$

(c) $S < Q < T$

(d) $Q = S$

(e) $Q = 0$

CPU Efficiency



Sum of lengths of all the green boxes should be equal to the length of blue box. Black box is context switch overhead, which is neither part of T nor Q .



$$\text{CPU efficiency} = \frac{\text{The useful CPU time}}{\text{Total CPU time}}$$

$$= \frac{\text{CPU_Burst}}{\text{CPU_Burst} + \text{Overhead_Per_Ctx} \times \# \text{Ctx_Switch}}$$

$$= \frac{T}{T + S \times \# \text{Ctx_Switch}}$$

We must include the first context switch to start executing the process, because this is also overhead.

Count #Ctx_Switch

	#Ctx_Switch	CPU Efficiency= $\frac{T}{T + S \times \#Ctx_Switch}$
(a) $Q = \infty$	1	$T/(S + T)$
(b) $Q > T$	1	$T/(S + T)$
(c) $S < Q < T$	$\text{Ceil}(T/Q)$	$T/(T + S * \text{ceil}(T/Q))$
(d) $Q = S$	$\text{Ceil}(T/S)$	$T/(T + S * \text{ceil}(T/S))$ approx 0.5
(e) $Q = 0$	Inf ∞	0